SELENIUM JAVA PROJECT INTEGRATION

WITH GITHUB ACTIONS

Brief Introduction to Selenium and TestNG

When building an automated testing pipeline, **Selenium** and **TestNG** serve two completely different but essential purposes:

- **Selenium (WebDriver):** This is the automation engine. It acts as the browser simulator, executing actions like opening a browser, locating HTML buttons, typing text, or scraping content. It has no logic to determine if a test passed, failed, or how to generate a report.
- **TestNG ("Test Next Generation"):** This is the Java testing framework. It orchestrates the tests, handles execution order, provides assertions to validate outcomes, and automatically outputs detailed test summaries.

They are not competitors; they are complementary tools used together in automated testing.

While Selenium acts as the "hands" that physically control the web browser, TestNG acts as the "brain" that organizes, executes, and reports on those tests.

Core Differences

Feature	**Selenium**	**TestNG**
Primary Purpose	**Browser Automation** (clicking, typing, navigating web pages).	**Test Management** (grouping, sequencing, validating results).
Type of Tool	API / Library Suite.	Testing Framework.
Language Support	Multiple languages (Java, Python, C#, Ruby, JavaScript).	Strictly **Java** (inspired by JUnit).

Reporting	No built-in reporting capabilities.	Generates built-in HTML test reports.
Assertions	Cannot validate if a test "passed" or "failed" on its own.	Provides Assert commands to check actual vs. expected results.
Execution Control	Executes code line-by-line via a standard main method.	Controls execution flow using **Annotations** (e.g., @Test, @BeforeMethod).

How They Work Together

If you only use **Selenium WebDriver**, you can write a script that opens Chrome and clicks a button, but you won't have a clean way to run 50 different tests in a specific order, handle failures gracefully, or get a clean report showing what failed.

When you combine them in a **Hybrid Framework**:

1. **TestNG** triggers the workflow. It uses an annotation like @BeforeMethod to launch the browser.
2. **Selenium** steps in to perform the actions inside the browser (e.g., logging into a website, adding an item to a cart).
3. **TestNG** evaluates the outcome using an assertion (e.g., verifying the user is on the dashboard) and logs whether the test passed or failed.
4. **TestNG** outputs a comprehensive report and handles any required cleanup, like closing the browser via @AfterMethod.

Why Use TestNG Instead of JUnit?

While JUnit is the standard for *unit testing* (testing internal Java code functions), TestNG was specifically designed to handle *functional, end-to-end integration testing*—exactly what you do when automating user actions on an HTML page.

Here is why TestNG is far better suited than JUnit for a **Maven project** interacting with HTML files:

1. Native HTML Test Reports

- **JUnit:** Provides basic XML output meant for a CI/CD server. If you want a readable visual document for stakeholders, you have to add extra plugins (like ExtentReports or Allure) to your pom.xml.
- **TestNG:** Automatically generates a comprehensive **HTML report** (index.html) directly inside your Maven project's test-output directory the moment your suite runs. It details precisely which HTML interactions passed, failed, or were skipped.

2. Advanced HTML Page Automation Lifecycle

Testing a web interface requires tight control over when a browser starts and stops. TestNG features a more robust, multi-layered annotation tier than JUnit:

- **@BeforeSuite / @AfterSuite**: Great for initializing global test databases or environments before any page testing begins.
- **@BeforeTest / @AfterTest**: Runs once per <test> block defined in an XML file.
- **@BeforeClass / @AfterClass**: Ideal for launching a shared browser window to interact with an HTML workflow, avoiding the overhead of launching a fresh browser for every single verification step.

3. Handling Dependent Elements and Sequential UI Workflows

Web-based interactions usually rely on a sequential flow (e.g., Step 1: Open Login page → Step 2: Click Login → Step 3: Check Dashboard).

- **JUnit:** Treats all tests as strictly independent. If your "Login" test fails, JUnit will still attempt to run the "Dashboard Check" test, causing a string of distracting, false-failure cascading errors.
- **TestNG:** Features built-in dependencies (@Test(dependsOnMethods = "loginTest")). If the prerequisite test fails, TestNG intelligently **skips** the downstream UI checks, keeping your test reports clean and accurate.

4. Explicit UI Execution Control (priority)

- **JUnit:** Relies heavily on method names or custom naming classes to enforce an execution order.
- **TestNG:** Provides an explicit priority attribute natively inside the code (e.g., @Test(priority = 1)) to ensure actions happen in the exact order a human user would click through the HTML document.

Experiment 6:

Design and execute automated functional test scripts using the Selenium WebDriver to validate web application UI components and verify expected user behaviors against real-world browser environments.

Step 1 – Create the Maven Java Project

Create or open a Maven Java project in Eclipse. Use the following structure. The original project contains a local HTML login page, a Selenium TestNG class, pom.xml and testng.xml.

```
SeleniumLocalWebTest/
│
├── pom.xml
├── testng.xml
├── webpage/
│   └── login.html
│
└── src/
    └── test/
        └── java/
            └── LocalLoginTest.java
```

Step 2 – Configure Selenium and TestNG in pom.xml

Use Maven dependencies for Selenium Java and TestNG. The versions below are retained from the original experiment.

```
</dependencies>
  <dependency>
    <groupId>org.seleniumhq.selenium</groupId>
    <artifactId>selenium-java</artifactId>
    <version>4.25.0</version>
  </dependency>
  <dependency>
```

```
    <groupId>org.testng</groupId>
    <artifactId>testng</artifactId>
    <version>7.10.2</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

Note: In default pom.xml generated, remove the dependencies for Junit and add selenium and testing dependencies

```
<dependencyManagement>                          -------→ Remove
  <dependencies>
   <dependency>
    <groupId>org.junit</groupId>
    <artifactId>junit-bom</artifactId>
    <version>5.11.0</version>
    <type>pom</type>
    <scope>import</scope>
   </dependency>
  </dependencies>
 </dependencyManagement>

 <dependencies>
  <dependency>
   <groupId>org.junit.jupiter</groupId>
   <artifactId>junit-jupiter-api</artifactId>
   <scope>test</scope>
  </dependency>
  <!-- Optionally: parameterized tests support -->
  <dependency>
   <groupId>org.junit.jupiter</groupId>
   <artifactId>junit-jupiter-params</artifactId>
   <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.seleniumhq.selenium</groupId>
    <artifactId>selenium-java</artifactId>
    <version>4.25.0</version>
  </dependency>                                 ------→ Add

  <dependency>
    <groupId>org.testng</groupId>
    <artifactId>testng</artifactId>
    <version>7.10.2</version>
    <scope>test</scope>
  </dependency>
  </dependencies>
```

Step 3 – Create the Local Web Page

Create webpage/login.html. The page contains a username field, password field, Login button and message area.

```
<!DOCTYPE html>
<html>
<head>
  <title>Student Login</title>
</head>
<body>
<div id="loginBox" style="background-color: lightblue; width: 300px; height: 200px">
  <h2>Student Login</h2>
  <input id="username" type="text" placeholder="Username"><br><br>
  <input id="password" type="password" placeholder="Password"><br><br>
  <button id="loginButton" onclick="login()">Login</button>
  <div id="message"></div>
</div>

<script>
function login() {
  var username = document.getElementById("username").value;
  var password = document.getElementById("password").value;

  if (username === "student" && password === "12345") {
    document.getElementById("message").innerText = "Login Successful";
  } else {
    document.getElementById("message").innerText =
      "Invalid Username or Password";
  }
}
</script>
</body>
</html>
```

Step 4 – Create the Selenium Test Class

Create LocalLoginTest.java. The tests open the local HTML page before each test. The class contains three tests: valid login, invalid login and background-color verification.

```
import java.io.File;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
```

```
import org.testng.Assert;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Test;

public class LocalLoginTest {
   WebDriver driver;

   @BeforeMethod
   public void setUp() {
      driver = new ChromeDriver();
      driver.manage().window().maximize();

      String page = new File("webpage/login.html")
            .getAbsoluteFile()
            .toURI()
            .toString();

      driver.get(page);
   }

   @Test
   public void validLoginTest() {
      driver.findElement(By.id("username")).sendKeys("student");
      driver.findElement(By.id("password")).sendKeys("12345");
      driver.findElement(By.id("loginButton")).click();

      String message = driver.findElement(By.id("message")).getText();
      System.out.println("Message: " + message);
      Assert.assertEquals(message, "Login Successful");
   }

   @Test
   public void invalidLoginTest() {
      driver.findElement(By.id("username")).sendKeys("wrong");
      driver.findElement(By.id("password")).sendKeys("wrong");
      driver.findElement(By.id("loginButton")).click();

      String message = driver.findElement(By.id("message")).getText();
      System.out.println("Message: " + message);
      Assert.assertEquals(message, "Invalid Username or Password");
   }

   @Test
   public void checkBackgroundColor() {
      String bgColor =
```

```
        driver.findElement(By.id("loginBox"))
            .getCssValue("background-color");

        System.out.println("Background Color: " + bgColor);
        Assert.assertEquals(bgColor, "rgba(173, 216, 230, 1)");
    }

    @AfterMethod
    public void tearDown() {
        driver.quit();
    }
}
```

Step 5 – Create testng.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">

<suite name="Selenium Test Suite">
    <test name="Local Selenium Tests">
        <classes>
            <class name="LocalLoginTest"/>
        </classes>
    </test>
</suite>
```

Step 6 – Test the Project Locally

Prerequisites Check : Ensure you have the **TestNG for Eclipse** plugin installed (if running directly from the IDE interface).

- Go to **Help** > **Eclipse Marketplace** > Search for **TestNG** > Install it if it isn't already installed.

Before pushing the project to GitHub, verify that the tests work in Eclipse.

- Open Eclipse.
- Right-click LocalLoginTest.java.
- Choose Run As → TestNG Test.
- Verify that Chrome opens and the three test cases execute.

- Expected result: 3 tests passed and 0 failures.

Step 7 – Create a GitHub Repository

- Sign in to GitHub.
- Click New repository.
- Enter a repository name such as SeleniumLocalWebTest.
- Choose the required visibility.
- Create the repository.

Step 8 – Initialize Git and Push the Project

Open Command Prompt or PowerShell in the project folder and run:

```
git init
git add .
git commit -m "Add Selenium Java TestNG project"
git branch -M main
git remote add origin https://github.com/<USERNAME>/SeleniumLocalWebTest.git
git push -u origin main
```

Step 9 – Create the GitHub Actions Workflow

Create the following folder structure in the project:

```
.github/
└── workflows/
    └── selenium-tests.yml
```

The workflow checks out the source code, installs Java, installs Maven dependencies, starts Google Chrome in headless mode through Chrome options, and runs the TestNG tests.

```
name: Selenium Java Tests

on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

jobs:
  selenium-test:
    runs-on: ubuntu-latest

    steps:
```

```
    - name: Checkout source code
      uses: actions/checkout@v4

    - name: Set up JDK 17
      uses: actions/setup-java@v4
      with:
        distribution: temurin
        java-version: '17'
        cache: maven

    - name: Set up Chrome
      uses: browser-actions/setup-chrome@v1

    - name: Verify Java and Chrome
      run: |
        java -version
        google-chrome --version

    - name: Run Selenium tests
      run: mvn clean test
```

Important – Chrome Headless Configuration

Because GitHub Actions runs on a cloud-hosted Ubuntu runner without a normal desktop, the Selenium test should use Chrome in headless mode. Modify the setUp() method as follows:

```
import org.openqa.selenium.chrome.ChromeOptions;

@BeforeMethod
public void setUp() {
   ChromeOptions options = new ChromeOptions();
   options.addArguments("--headless=new");
   options.addArguments("--no-sandbox");
   options.addArguments("--disable-dev-shm-usage");
   options.addArguments("--window-size=1920,1080");

   driver = new ChromeDriver(options);

   String page = new File("webpage/login.html")
         .getAbsoluteFile()
         .toURI()
         .toString();

   driver.get(page);
}
```

Step 10 – Commit the GitHub Actions Workflow

git add .
git commit -m "Add GitHub Actions Selenium workflow"
git push origin main

Step 11 – Check GitHub Actions

- Open the GitHub repository in the browser.
- Click the Actions tab.
- Select Selenium Java Tests.
- Open the latest workflow run.
- Open the selenium-test job.
- Check the Run Selenium tests step.

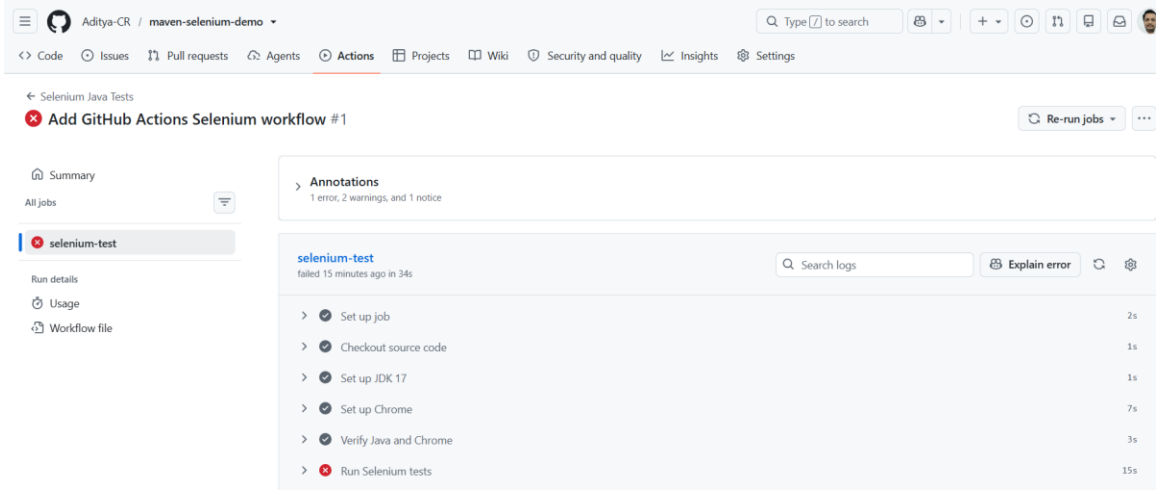
Step 12 – Expected GitHub Actions Result

A successful workflow should show:

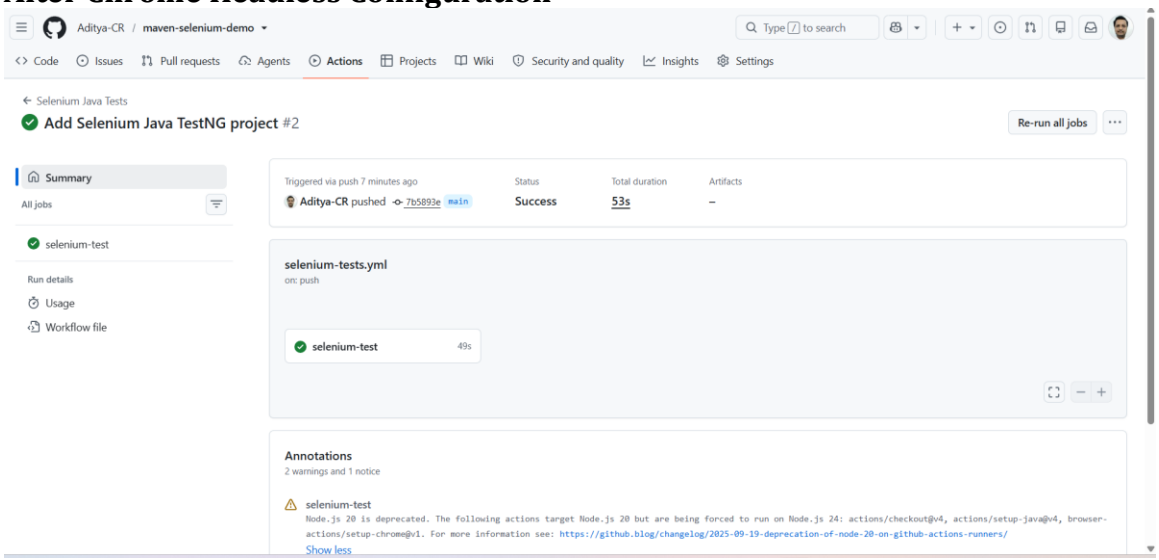
✓ Checkout source code
✓ Set up JDK 17
✓ Set up Chrome
✓ Verify Java and Chrome
✓ Run Selenium tests

Tests run: 3
Failures: 0
Skips: 0

Without Chrome Headless Configuration



After Chrome Headless Configuration



Step 13 – Complete CI Flow

Developer
↓
Write Selenium + Java + TestNG code
↓
Run tests locally
↓
git add / commit / push
↓
GitHub Repository
↓
GitHub Actions Workflow
↓
Checkout code
↓
Install JDK + Chrome
↓
mvn clean test
↓
Selenium tests execute
↓
PASS / FAIL
↓
GitHub Actions result

Note-1: LocalLoginTest.java

Placing LocalLoginTest.java inside src/test/java/com/example/ (or another package path) is strongly recommended over placing it directly under src/test/java.

Why a Package Structure is Preferred
Avoids Default Package Issues: Files directly under src/test/java belong to Java's "default package". Classes in the default package cannot be cleanly imported or accessed by other packaged classes, which often causes compilation and execution issues in IDEs like IntelliJ or Eclipse.

Maven Conventions: Maven and TestNG expect Java code to be structured into packages (e.g., com.example.tests or com.project.login).

TestNG Configuration: Defining package names makes your testng.xml suite configurations much cleaner to manage and scale.

Note-2: login.html

The project structure we have right now is functionally valid, but placing login.html inside src/main/java done in previous Maven Experiment-3 is not recommended.

In Maven standard directory layout:

- src/main/java is strictly for Java source code (.java files). Non-Java files placed here are generally ignored or excluded during compilation unless explicitly configured as resources.

- src/main/resources is the standard Maven directory intended for static files, templates, test assets, and HTML files.

The cleanest and most conventional way to structure your project is :

SeleniumLocalWebTest/
├── pom.xml
├── testng.xml
└── **src/**
  ├── **main/**
  │ └── **resources/**
  │ └── webpage/
  │ └── login.html **<-- Recommended location for web assets**
  └── test/

└── java/
└── LocalLoginTest.java

Key Considerations

1. **Root Directory (**webpage/login.html**)**: Keeping webpage/ at the root level (as in your current setup) works fine for quick local test suites, provided you load it in Selenium using an absolute file path or relative project path like file:///" + System.getProperty("user.dir") + "/webpage/login.html.

2. **src/main/resources/** : Moving login.html here ensures it gets packaged into your classpath automatically during Maven builds, making test paths consistent across different environments (CI/CD pipelines, local runs, etc.).

Changes to be made if login.html is kept inside src/main/resources/webpage :

Option 1:

```
public void setUp() {
driver = new ChromeDriver();
driver.manage().window().maximize();
String page = new File("src/main/resources/webpage/login.html")
.getAbsoluteFile() .toURI() .toString();
driver.get(page);
}
```

Option 2:

```
import java.net.URL;

@BeforeMethod
public void setUp() {
   driver = new ChromeDriver();
   driver.manage().window().maximize();

   // Fetches the resource relative to target/classes/
   URL resource = getClass().getClassLoader().getResource("webpage/login.html");

   if (resource == null) {
      throw new IllegalArgumentException("File not found in resources!");
   }

   driver.get(resource.toString());
}
```

Additional Experiment:
Containerising & Publishing to Docker Hub

Objective
To modify the existing automated pipeline so that every successful test run triggers the creation of a isolated **Docker image** containing your test framework, which is then securely pushed to your public/private **Docker Hub repository** upon a successful build.

Step 1: Create a Docker Hub Account & Repository
Before updating your code, you need a destination to host your container image.

1. Go to Docker Hub and sign in (or create an account).
2. Go to MyHub and Click **Create Repository**.
3. Name your repository selenium-testng-automation.
4. Set the visibility to **Public** and click **Create**.

Step 2: Generate a Docker Hub Access Token
For safety, you should never store your raw password in GitHub. Instead, use an Access Token.

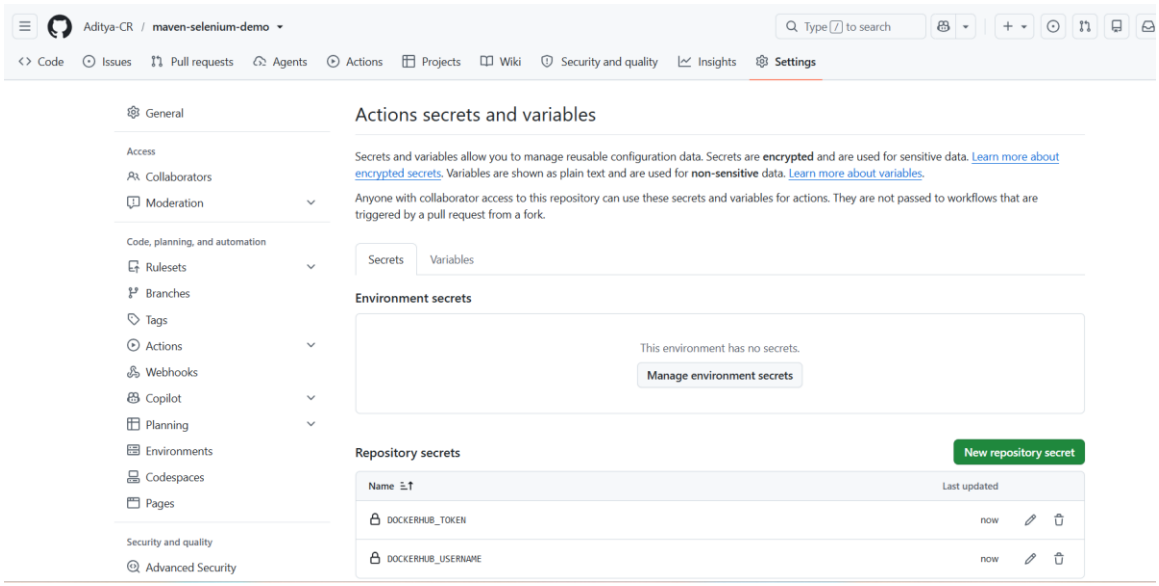
1. In Docker Hub, click on your profile avatar in the top-right corner and select **Account Settings**.
2. Navigate to **Security** in the left sidebar.
3. Click **Personal Access Tokens → Generate New Token**.
4. Name it GitHub-Actions-CI and grant it **Read & Write** permissions.
5. **Copy the token immediately** (you will not see it again).

docker login -u dradityacr
dckr_pat_23j8bk-pMiUDEEvXyaBcTz7eXDA

Step 3: Save Credentials as GitHub Secrets

1. Open your **SeleniumLocalWebTest** repository on GitHub.
2. Navigate to **Settings → Secrets and variables → Actions**.
3. Click **New repository secret** and add the following two secrets:
    - **Name:** DOCKERHUB_USERNAME
      Value: *Your actual Docker Hub username (eg : dradityacr)*

Name: DOCKERHUB_TOKEN
Value: *The personal access token you generated in Step 2 (eg:* dckr_pat_23j8k-pMiUDvXyaAcTz7eND)



Step 4: Create a Dockerfile in the Project Root
To bundle your Java project, you need a recipe blueprint file. Create a file named exactly Dockerfile (no extension) at the root level of your project structure:

SeleniumLocalWebTest/
│ ----- **Dockerfile**
├── pom.xml
├── testng.xml
├── webpage/
│ └── login.html
│
└── src/
└── test/
└── java/
└── LocalLoginTest.java

Dockerfile Code

Step 1: Use an official Maven image with JDK 17 to build and bundle the app
FROM maven:3.8.6-openjdk-17-slim AS build
WORKDIR /app

Copy the build configuration and source code
COPY pom.xml .
COPY src ./src

Pre-fetch dependencies to speed up subsequent image builds

```
RUN mvn dependency:go-offline -B

# Package the application tests without executing them during image compilation
RUN mvn test-compile

# Step 2: Use a slim runtime image containing Chrome for execution headless environment
FROM openjdk:17-slim
WORKDIR /app

# Install stable Chrome dependencies required for headless Selenium runs
RUN apt-get update && apt-get install -y \
   wget \
   gnupg \
   curl \
   unzip \
   libglib2.0-0 \
   libnss3 \
   libgconf-2-4 \
   libfontconfig1 \
   chromium \
   && rm -rf /var/lib/apt/lists/*

# Copy compiled classes and dependencies from the build stage
COPY --from=build /app /app

# Set environment variable to target the internal Chromium binary if needed
ENV CHROME_BIN=/usr/bin/chromium

# Default command to run the TestNG tests inside the container
CMD ["mvn", "test"]
```

Step 5: Extend the GitHub Actions Workflow File

Modify your .github/workflows/selenium-tests.yml file to include your Docker authentication, building steps, and target deployment push tasks:

```
name: Selenium Java Tests & Docker Publish

on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

jobs:
```

```
selenium-test:
  runs-on: ubuntu-latest
  steps:
    - name: Checkout source code
      uses: actions/checkout@v4

    - name: Set up JDK 17
      uses: actions/setup-java@v4
      with:
        distribution: 'temurin'
        java-version: '17'
        cache: maven

    - name: Set up Chrome
      uses: browser-actions/setup-chrome@v1

    - name: Verify Java and Chrome
      run: |
        java -version
        google-chrome --version

    - name: Run Selenium tests
      run: mvn clean test

# NEW JOB: Executes sequentially only after validation tests pass perfectly
build-and-publish-docker:
  needs: selenium-test
  runs-on: ubuntu-latest
  if: github.event_name == 'push' # Only push to registry on formal merges/pushes
  steps:
    - name: Checkout source code
      uses: actions/checkout@v4

    - name: Log in to Docker Hub
      uses: docker/login-action@v3
      with:
        username: ${{ secrets::DOCKERHUB_USERNAME }}
        password: ${{ secrets::DOCKERHUB_TOKEN }}

    - name: Set up Docker Buildx
      uses: docker/setup-buildx-action@v3

    - name: Build and Push Docker Image
      uses: docker/build-push-action@v5
      with:
        context: .
```

```
        file: ./Dockerfile
        push: true
        tags: |
          ${{ secrets.DOCKERHUB_USERNAME }}/selenium-testng-automation:latest
          ${{ secrets.DOCKERHUB_USERNAME }}/selenium-testng-automation:${{
github.sha }}
```

Step 6: Commit and Push Changes

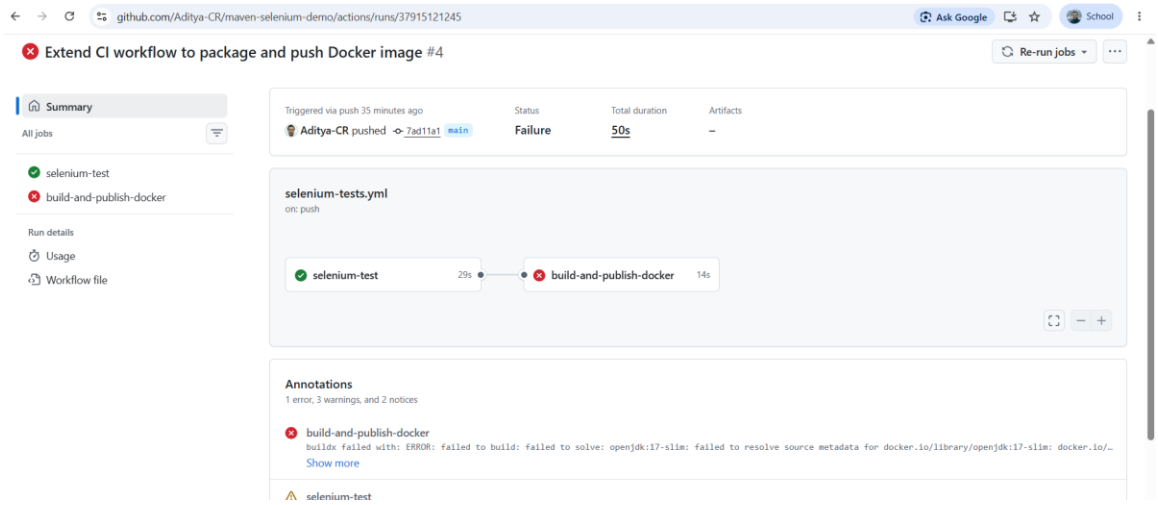
Save your files locally and use git commands to deploy the updates:
Git add . (or) git add Dockerfile .github/workflows/selenium-tests.yml
git commit -m "Extend CI workflow to package and push Docker image"
git push origin main

Step 7: Verification Checklist

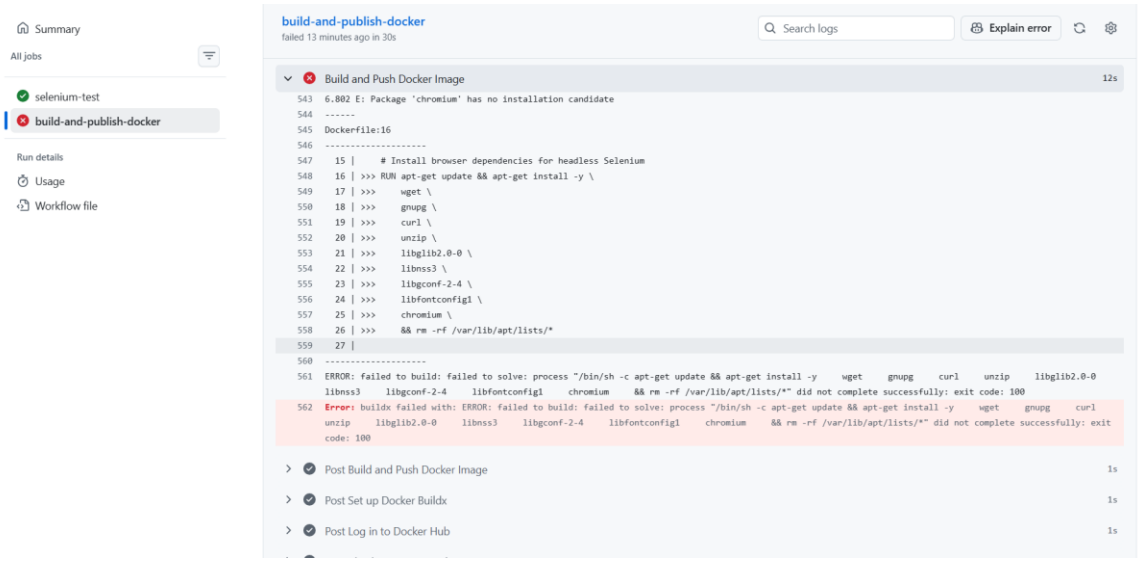
1. **GitHub Actions:** Open the **Actions** tab in your GitHub repository. You will see a newly extended two-stage matrix workflow pipeline: selenium-test → build-and-publish-docker.
2. **Docker Hub Registry:** Open your Docker Hub dashboard profile page. Under the selenium-testng-automation repository tags section, you will see a latest tag timestamped right now, along with a secondary tag representing your specific unique Git commit hash.

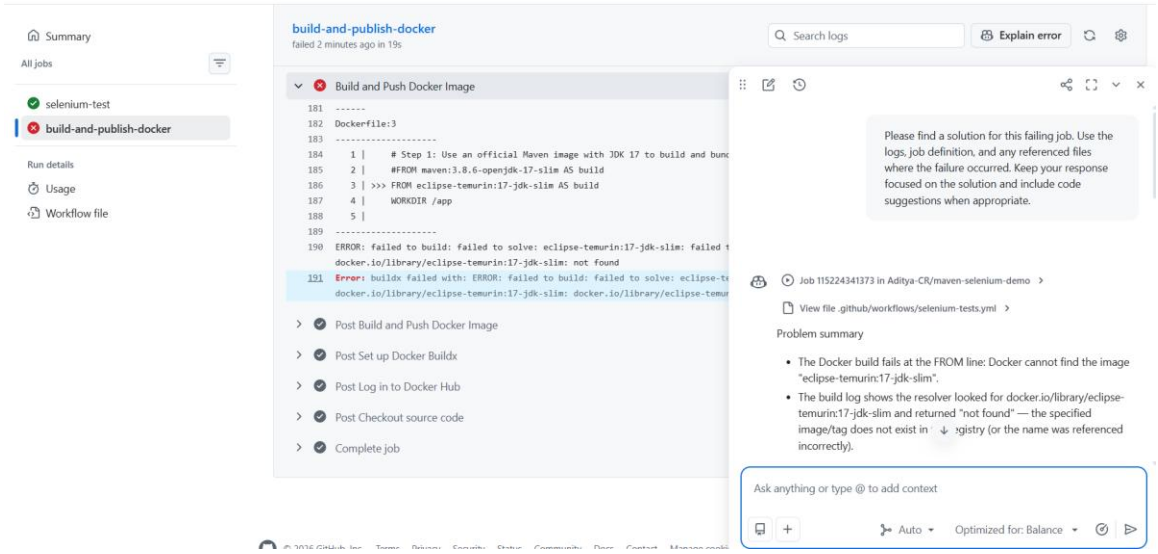
Output

Selenium Test is passed, However the docker image build and publishing to docker hub have failed



Inspect the error, and rectify it.
Click on → "**Explain Error**" to get **AI assistance** and fix the error





Update the Dockerfile and commit changes in local system and push to github
Or
Update the Dockerfile and commit changes in github itself

```
FROM maven:3.9.6-eclipse-temurin-17 AS build
WORKDIR /app

# Copy the build config first so Maven can cache dependencies
COPY pom.xml .
RUN mvn dependency:go-offline -B

# Copy source and compile
COPY src ./src
RUN mvn test-compile -B

FROM eclipse-temurin:17-jre-jammy AS runtime
WORKDIR /app

# Install browser dependencies and Google Chrome (stable)
RUN apt-get update && apt-get install -y \
  wget \
  gnupg \
  curl \
  unzip \
  libglib2.0-0 \
  libnss3 \
  libgconf-2-4 \
  libfontconfig1 \
  ca-certificates \
  fonts-liberation \
  && rm -rf /var/lib/apt/lists/* \
  && curl -fsSL https://dl.google.com/linux/linux_signing_key.pub | gpg --dearmor -o
/usr/share/keyrings/google-linux-signing-keyring.gpg \
  && echo "deb [arch=amd64 signed-by=/usr/share/keyrings/google-linux-signing-keyring.gpg]
http://dl.google.com/linux/chrome/deb/ stable main" > /etc/apt/sources.list.d/google-chrome.list \
```

```
    && apt-get update && apt-get install -y google-chrome-stable \
    && rm -rf /var/lib/apt/lists/*

# Copy only the built app from the build stage
COPY --from=build /app/target /app/target

ENV CHROME_BIN=/usr/bin/google-chrome-stable

# If this image is meant to run tests inside the container, keep this:
CMD ["mvn", "test"]
```

Output

Github Workflow successfully executed

